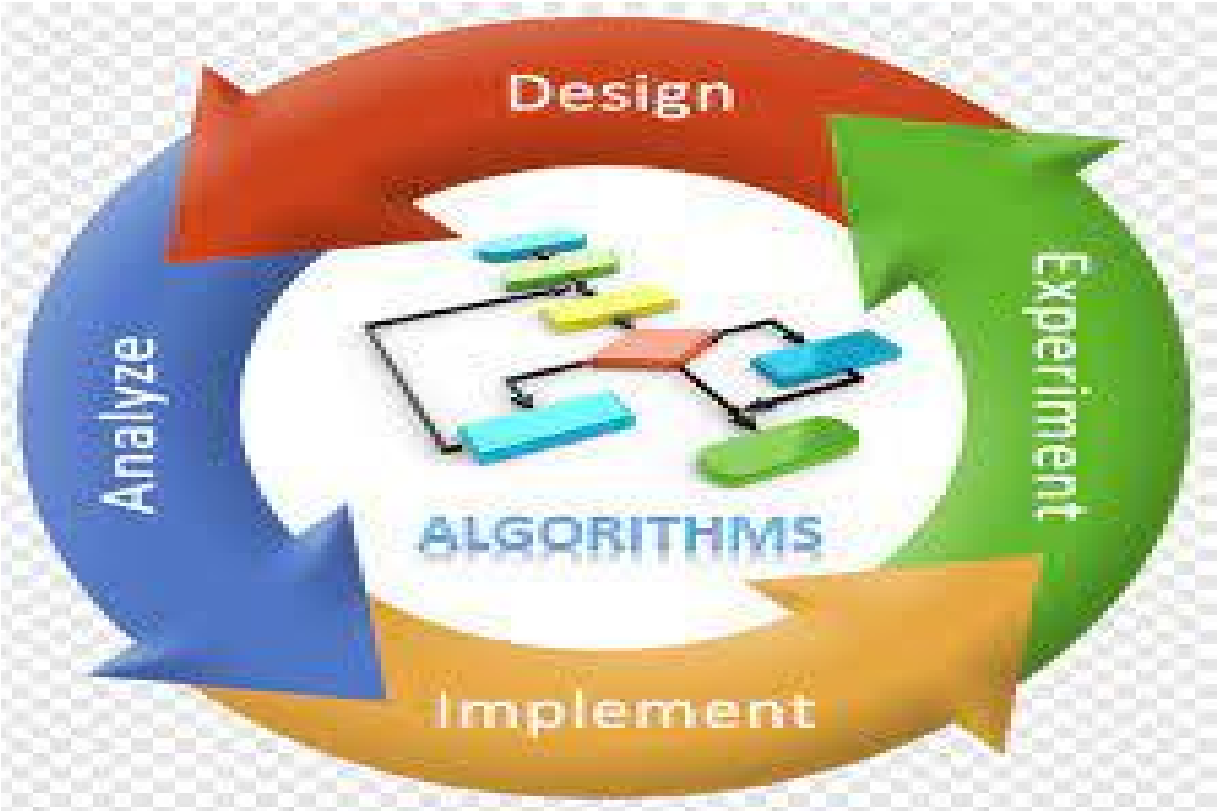
`

CMR INSTITUTE OF TECHNOLOGY

Department of Computer Science and Engineering

Analysis and Design of Algorithms Manual

(BCSL404)



EXPERIMENTS

1 Design and implement a C/C++ Program to find Minimum Cost Spanning Tree of a given connected undirected graph using Kruskal's algorithm.

2 Design and implement C/C++ Program to find Minimum Cost Spanning Tree of a given connected undirected graph using Prim's algorithm.

3 a. Design and implement C/C++ Program to solve All-Pairs Shortest Paths problem using Floyd's algorithm.

b. Design and implement C/C++ Program to find the transitive closure using Warshal's algorithm.

4 Design and implement C/C++ Program to find shortest paths from a given vertex in a weighted connected graph to other vertices using Dijkstra's algorithm.

5 Design and implement C/C++ Program to obtain the Topological ordering of vertices in a given digraph.

6 Design and implement C/C++ Program to solve 0/1 Knapsack problem using Dynamic Programming method.

7 Design and implement C/C++ Program to solve discrete Knapsack and continuous Knapsack problems using greedy approximation method.

8 Design and implement C/C++ Program to find a subset of a given set S = {sl , s2,.....,sn} of n positive integers whose sum is equal to a given positive integer d.

9. Design and implement C/C++ Program to sort a given set of n integer elements using Selection Sort method and compute its time complexity. Run the program for varied values of n> 5000 and record the time taken to sort. Plot a graph of the time taken versus n. The elements can be read from a file or can be generated using the random number generator.

10 Design and implement C/C++ Program to sort a given set of n integer elements using Quick Sort method and compute its time complexity. Run the program for varied values of n> 5000 and record the time taken to sort. Plot a graph of the time taken versus n. The elements can be read from a file or can be generated using the random number generator.

11 Design and implement C/C++ Program to sort a given set of n integer elements using Merge Sort method and compute its time complexity. Run the program for varied values of n> 5000, and record the time taken to sort. Plot a graph of the time taken versus n. The elements can be read from a file or can be generated using the random number generator.

12 Design and implement C/C++ Program for N Queens problem using Backtracking.

1. **Design and implement C/C++ Program to sort a given set of n integer elements using Selection Sort method and compute its time complexity. Run the program for varied values of n> 5000 and record the time taken to sort. Plot a graph of the time taken versus n. The elements can be read from a file or can be generated using the random number generator.**

```
#include <stdio.h>
#include<stdlib.h>
#include<time.h>

void selectionSort(int arr[], int n)
{
        int i, j, min_idx,temp;

        // One by one move boundary of unsorted subarray
        for (i = 0; i < n-1; i++)
        {
                // Find the minimum element in unsorted array
                min_idx = i;
                for (j = i+1; j < n; j++)
                {
                   if (arr[j] < arr[min_idx])
                           min_idx = j;
                }
                // Swap the found minimum element with the first element
                if(min_idx != i)
                {
                        temp=arr[min_idx];
                        arr[min_idx]=arr[i];
                        arr[i]=temp;
                }

        }
}

/* Function to print an array */
void printArray(int arr[], int size)
{
```

```
        int i;
        for (i=0; i < size; i++)
                printf("%d ", arr[i]);
        printf("\n");
}

//  program to test above functions
int main()
{   int i,n;
    clock_t start, end;
    double cpu_time_used;

        printf("Enter the value of n:");
    scanf("%d",&n);
    int arr[n];
    for(i=0;i<n;i++)
    {

      arr[i]=rand()%1000;
      printf("%d ",arr[i]);
    }
    start=clock();
    selectionSort(arr, n);
    end=clock();
    cpu_time_used = ((double) (end - start)) / CLOCKS_PER_SEC;

        printf("Sorted array: \n");
        printArray(arr, n);

    printf("Execution time: %f seconds\n", cpu_time_used);

        return 0;
}
```

Sample Output:

```
Enter the value of n:100
383 886 777 915 793 335 386 492 649 421 362 27 690 59 763 926 540 426 172 736 211 368 567 429
782 530 862 123 67 135 929 802 22 58 69 167 393 456 11 42 229 373 421 919 784 537 198 324 315 370
413 526 91 980 956 873 862 170 996 281 305 925 84 327 336 505 846 729 313 857 124 895 582 545
```

814 367 434 364 43 750 87 808 276 178 788 584 403 651 754 399 932 60 676 368 739 12 226 586 94 539 Sorted array:
11 12 22 27 42 43 58 59 60 67 69 84 87 91 94 123 124 135 167 170 172 178 198 211 226 229 276 281 305 313 315 324 327 335 336 362 364 367 368 368 370 373 383 386 393 399 403 413 421 421 426 429 434 456 492 505 526 530 537 539 540 545 567 582 584 586 649 651 676 690 729 736 739 750 754 763 777 782 784 788 793 802 808 814 846 857 862 862 873 886 895 915 919 925 926 929 932 956 980 996
Execution time: 0.000019 seconds

2. Design and implement C/C++ Program to sort a given set of n integer elements using Quick Sort method and compute its time complexity. Run the program for varied values of n> 5000 and record the time taken to sort. Plot a graph of the time taken versus n. The elements can be read from a file or can be generated using the random number generator.

```
#include <stdio.h>
#include<stdlib.h>
#include<time.h>

// Function to swap two elements
void swap(int* a, int* b)
{
   int temp = *a;
   *a = *b;
   *b = temp;
}

// Partition function
int partition(int arr[], int low, int high)
{

   // initialize pivot to be the first element
   int pivot = arr[low];
   int i = low;
   int j = high;

   while (i < j) {

      // condition 1: find the first element greater than
      // the pivot (from starting)
      while (arr[i] <= pivot && i <= high - 1) {
         i++;
      }

      // condition 2: find the first element smaller than
      // the pivot (from last)
      while (arr[j] > pivot && j >= low + 1) {
         j--;
      }
```

```
        if (i < j) {
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[low], &arr[j]);
    return j;
}

// QuickSort function
void quickSort(int arr[], int low, int high)
{
    if (low < high) {

        // call Partition function to find Partition Index
        int partitionIndex = partition(arr, low, high);

        // Recursively call quickSort() for left and right
        // half based on partition Index
        quickSort(arr, low, partitionIndex - 1);
        quickSort(arr, partitionIndex + 1, high);
    }
}

// driver code
int main()
{
  int i,n;
    clock_t start, end;
    double cpu_time_used;

            printf("Enter the value of n:");
    scanf("%d",&n);
    int arr[n];
    for(i=0; i<n; i++)
    {

        arr[i]=rand()%1000;
        printf("%d ",arr[i]);
    }
    start=clock();
```

```
    // calling quickSort() to sort the given array
    quickSort(arr, 0, n - 1);
    end=clock();
    cpu_time_used = ((double) (end - start)) / CLOCKS_PER_SEC;

    // printing the sorted array
    printf("\nSorted array: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }


    printf("\nExecution time: %f seconds\n", cpu_time_used);


    return 0;
}
```

Output

Enter the value of n:500

383 886 777 915 793 335 386 492 649 421 362 27 690 59 763 926 540 426 172 736 211 368 567 429 782 530 862 123 67 135 929 802 22 58 69 167 393 456 11 42 229 373 421 919 784 537 198 324 315 370 413 526 91 980 956 873 862 170 996 281 305 925 84 327 336 505 846 729 313 857 124 895 582 545 814 367 434 364 43 750 87 808 276 178 788 584 403 651 754 399 932 60 676 368 739 12 226 586 94 539 795 570 434 378 467 601 97 902 317 492 652 756 301 280 286 441 865 689 444 619 440 729 31 117 97 771 481 675 709 927 567 856 497 353 586 965 306 683 219 624 528 871 732 829 503 19 270 368 708 715 340 149 796 723 618 245 846 451 921 555 379 488 764 228 841 350 193 500 34 764 124 914 987 856 743 491 227 365 859 936 432 551 437 228 275 407 474 121 858 395 29 237 235 793 818 428 143 11 928 529 776 404 443 763 613 538 606 840 904 818 128 688 369 917 917 996 324 743 470 183 490 499 772 725 644 590 505 139 954 786 669 82 542 464 197 507 355 804 348 611 622 828 299 343 746 568 340 422 311 810 605 801 661 730 878 305 320 736 444 626 522 465 708 416 282 258 924 637 62 624 600 36 452 899 379 550 468 71 973 131 881 930 933 894 660 163 199 981 899 996 959 773 813 668 190 95 926 466 84 340 90 684 376 542 936 107 445 756 179 418 887 412 348 172 659 9 336 210 342 587 206 301 713 372 321 255 819 599 721 904 939 811 940 667 705 228 127 150 984 658 920 224 422 269 396 81 630 84 292 972 672 850 625 385 222 299 640 42 898 713 298 190 524 590 209 581 819 336 732 155 994 4 379 769 273 776 850 255 860 142 579 884 993 205 621 567 504 613 961 754 326 259 944 202 202 506 784 21 842 868 528 189 872 908 958 498 36 808 753 248 303 333 133 648 890 754 567 746 368 529 500 46 788 797 249 990 303 33 363 497 253 892 686 125 152 996 975 188 157 729 436 460 414 921 460 304 28 27 50 748 556 902 794 697 699 43 39 2 428 403 500 681 647 538

159 151 535 134 339 692 215 127 504 629 49 964 285 429 343 335 177 900 238 971 949 289 367 988 292 795 743 144 829 390 682 340 541 569 826 232
Sorted array: 2 4 9 11 11 12 19 21 22 27 27 28 29 31 33 34 36 36 39 42 42 43 43 46 49 50 58 59 60 62 67 69 71 81 82 84 84 84 87 90 91 94 95 97 97 107 117 121 123 124 124 125 127 127 128 131 133 134 135 139 142 143 144 149 150 151 152 155 157 159 163 167 170 172 172 177 178 179 183 188 189 190 190 193 197 198 199 202 202 205 206 209 210 211 215 219 222 224 226 227 228 228 228 229 232 235 237 238 245 248 249 253 255 255 258 259 269 270 273 275 276 280 281 282 285 286 289 292 292 298 299 299 301 301 303 303 304 305 305 306 311 313 315 317 320 321 324 324 326 327 333 335 335 336 336 336 339 340 340 340 340 342 343 343 348 348 350 353 355 362 363 364 365 367 367 368 368 368 368 369 370 372 373 376 378 379 379 379 383 385 386 390 393 395 396 399 403 403 404 407 412 413 414 416 418 421 421 422 422 426 428 428 429 429 432 434 434 436 437 440 441 443 444 444 445 451 452 456 460 460 464 465 466 467 468 470 474 481 488 490 491 492 492 497 497 498 499 500 500 500 503 504 504 505 505 506 507 522 524 526 528 528 529 529 530 535 537 538 538 539 540 541 542 542 545 550 551 555 556 567 567 567 567 568 569 570 579 581 582 584 586 586 587 590 590 599 600 601 605 606 611 613 613 618 619 621 622 624 624 625 626 629 630 637 640 644 647 648 649 651 652 658 659 660 661 667 668 669 672 675 676 681 682 683 684 686 688 689 690 692 697 699 705 708 708 709 713 713 715 721 723 725 729 729 729 730 732 732 736 736 739 743 743 743 746 746 748 750 753 754 754 754 756 756 763 763 764 764 769 771 772 773 776 776 777 782 784 784 786 788 788 793 793 794 795 795 796 797 801 802 804 808 808 810 811 813 814 818 818 819 819 826 828 829 829 840 841 842 846 846 850 850 856 856 857 858 859 860 862 862 865 868 871 872 873 878 881 884 886 887 890 892 894 895 898 899 899 900 902 902 904 904 908 914 915 917 917 919 920 921 921 924 925 926 926 927 928 929 930 932 933 936 936 939 940 944 949 954 956 958 959 961 964 965 971 972 973 975 980 981 984 987 988 990 993 994 996 996 996 996
Execution time: 0.000060 seconds

3.Design and implement C/C++ Program to sort a given set of n integer elements using Merge Sort method and compute its time complexity. Run the program for varied values of n> 5000, and record the time taken to sort. Plot a graph of the time taken versus n. The elements can be read from a file or can be generated using the random number generator.

```
#include <stdio.h>
#include <stdlib.h>
#include<time.h>

// Merges two subarrays of arr[].
// First subarray is arr[l..m]
// Second subarray is arr[m+1..r]
void merge(int arr[], int l, int m, int r)
{
	int i, j, k;
	int n1 = m - l + 1;
	int n2 = r - m;

	// Create temp arrays
	int L[n1], R[n2];

	// Copy data to temp arrays
	// L[] and R[]
	for (i = 0; i < n1; i++)
		L[i] = arr[l + i];
	for (j = 0; j < n2; j++)
		R[j] = arr[m + 1 + j];

	// Merge the temp arrays back
	// into arr[l..r]
	// Initial index of first subarray
	i = 0;

	// Initial index of second subarray
	j = 0;

	// Initial index of merged subarray
	k = l;
	while (i < n1 && j < n2) {
		if (L[i] <= R[j]) {
			arr[k] = L[i];
			i++;
		}
		else {
```

```
                arr[k] = R[j];
                j++;
            }
            k++;
        }

        // Copy the remaining elements
        // of L[], if there are any
        while (i < n1) {
            arr[k] = L[i];
            i++;
            k++;
        }

        // Copy the remaining elements of
        // R[], if there are any
        while (j < n2) {
            arr[k] = R[j];
            j++;
            k++;
        }
}

// l is for left index and r is
// right index of the sub-array
// of arr to be sorted
void mergeSort(int arr[], int l, int r)
{
        if (l < r) {
            // Same as (l+r)/2, but avoids
            // overflow for large l and r
            int m = l + (r - l) / 2;

            // Sort first and second halves
            mergeSort(arr, l, m);
            mergeSort(arr, m + 1, r);

            merge(arr, l, m, r);
        }
}
```

```
// Function to print an array
void printArray(int A[], int size)
{
        int i;
        for (i = 0; i < size; i++)
                printf("%d ", A[i]);
        printf("\n");
}


int main()
{
         int i,n;
   clock_t start, end;
   double cpu_time_used;

        printf("Enter the value of n:");
   scanf("%d",&n);
   int arr[n];
   for(i=0;i<n;i++)
   {

      arr[i]=rand()%1000;
      printf("%d ",arr[i]);
   }
   start=clock();


   mergeSort(arr, 0, n - 1);
   end=clock();
   cpu_time_used = ((double) (end - start)) / CLOCKS_PER_SEC;

   // printing the sorted array
   printf("\nSorted array: ");
   for (int i = 0; i < n; i++) {
      printf("%d ", arr[i]);
   }
```

```
    printf("\nExecution time: %f seconds\n", cpu_time_used);

        return 0;
}
```

OUTPUT

Enter the value of n:10
383 886 777 915 793 335 386 492 649 421
Sorted array: 335 383 386 421 492 649 777 793 886 915
Execution time: 0.000003 seconds

Enter the value of n:500
383 886 777 915 793 335 386 492 649 421 362 27 690 59 763 926 540 426 172 736 211 368 567 429 782 530 862 123 67 135 929 802 22 58 69 167 393 456 11 42 229 373 421 919 784 537 198 324 315 370 413 526 91 980 956 873 862 170 996 281 305 925 84 327 336 505 846 729 313 857 124 895 582 545 814 367 434 364 43 750 87 808 276 178 788 584 403 651 754 399 932 60 676 368 739 12 226 586 94 539 795 570 434 378 467 601 97 902 317 492 652 756 301 280 286 441 865 689 444 619 440 729 31 117 97 771 481 675 709 927 567 856 497 353 586 965 306 683 219 624 528 871 732 829 503 19 270 368 708 715 340 149 796 723 618 245 846 451 921 555 379 488 764 228 841 350 193 500 34 764 124 914 987 856 743 491 227 365 859 936 432 551 437 228 275 407 474 121 858 395 29 237 235 793 818 428 143 11 928 529 776 404 443 763 613 538 606 840 904 818 128 688 369 917 917 996 324 743 470 183 490 499 772 725 644 590 505 139 954 786 669 82 542 464 197 507 355 804 348 611 622 828 299 343 746 568 340 422 311 810 605 801 661 730 878 305 320 736 444 626 522 465 708 416 282 258 924 637 62 624 600 36 452 899 379 550 468 71 973 131 881 930 933 894 660 163 199 981 899 996 959 773 813 668 190 95 926 466 84 340 90 684 376 542 936 107 445 756 179 418 887 412 348 172 659 9 336 210 342 587 206 301 713 372 321 255 819 599 721 904 939 811 940 667 705 228 127 150 984 658 920 224 422 269 396 81 630 84 292 972 672 850 625 385 222 299 640 42 898 713 298 190 524 590 209 581 819 336 732 155 994 4 379 769 273 776 850 255 860 142 579 884 993 205 621 567 504 613 961 754 326 259 944 202 202 506 784 21 842 868 528 189 872 908 958 498 36 808 753 248 303 333 133 648 890 754 567 746 368 529 500 46 788 797 249 990 303 33 363 497 253 892 686 125 152 996 975 188 157 729 436 460 414 921 460 304 28 27 50 748 556 902 794 697 699 43 39 2 428 403 500 681 647 538 159 151 535 134 339 692 215 127 504 629 49 964 285 429 343 335 177 900 238 971 949 289 367 988 292 795 743 144 829 390 682 340 541 569 826 232
Sorted array: 2 4 9 11 11 12 19 21 22 27 27 28 29 31 33 34 36 36 39 42 42 43 43 46 49 50 58 59 60 62 67 69 71 81 82 84 84 84 87 90 91 94 95 97 97 107 117 121 123 124 124 125 127 127 128 131 133 134 135 139 142 143 144 149 150 151 152 155 157 159 163 167 170 172 172 177 178 179 183 188 189 190 190 193 197 198 199 202 202 205 206 209 210 211 215 219 222 224 226 227 228 228 228 229 232 235

237 238 245 248 249 253 255 255 258 259 269 270 273 275 276 280 281 282 285 286 289 292 292 298
299 299 301 301 303 303 304 305 305 306 311 313 315 317 320 321 324 324 326 327 333 335 335 336
336 336 339 340 340 340 340 342 343 343 348 348 350 353 355 362 363 364 365 367 367 368 368 368
368 369 370 372 373 376 378 379 379 379 383 385 386 390 393 395 396 399 403 403 404 407 412 413
414 416 418 421 421 422 422 426 428 428 429 429 432 434 434 436 437 440 441 443 444 444 445 451
452 456 460 460 464 465 466 467 468 470 474 481 488 490 491 492 492 497 497 498 499 500 500 500
503 504 504 505 505 506 507 522 524 526 528 528 529 529 530 535 537 538 538 539 540 541 542 542
545 550 551 555 556 567 567 567 567 568 569 570 579 581 582 584 586 586 587 590 590 599 600 601
605 606 611 613 613 618 619 621 622 624 624 625 626 629 630 637 640 644 647 648 649 651 652 658
659 660 661 667 668 669 672 675 676 681 682 683 684 686 688 689 690 692 697 699 705 708 708 709
713 713 715 721 723 725 729 729 729 730 732 732 736 736 739 743 743 743 746 746 748 750 753 754
754 754 756 756 763 763 764 764 769 771 772 773 776 776 777 782 784 784 786 788 788 793 793 794
795 795 796 797 801 802 804 808 808 810 811 813 814 818 818 819 819 826 828 829 829 840 841 842
846 846 850 850 856 856 857 858 859 860 862 862 865 868 871 872 873 878 881 884 886 887 890 892
894 895 898 899 899 900 902 902 904 904 908 914 915 917 917 919 920 921 921 924 925 926 926 927
928 929 930 932 933 936 936 939 940 944 949 954 956 958 959 961 964 965 971 972 973 975 980 981
984 987 988 990 993 994 996 996 996 996
Execution time: 0.000062 seconds

Program 2: Design and implement C/C++ Program to find Minimum Cost Spanning Tree of a given connected undirected graph using Prim's algorithm.

```
#include <stdio.h>

void my_prim(int adj[][10], int N) {
    int i, j, nv, min, min_cost = 0, u = 0, v = 0;
```

```
    int visit[10] = {0}; // Initialize all elements to 0

    visit[0] = 1; // Start from node 0
    nv = 1;

    while (nv < N) {
        min = 999;

        for (i = 0; i < N; i++) {
            if (visit[i] == 1) { // If visited
                for (j = 0; j < N; j++) {
                    if (adj[i][j] < min) {
                        min = adj[i][j];


                        u = i;
                        v = j;

                    }
                }
            }
        }
        adj[u][v] = 999;
        adj[v][u] = 999;
        if (visit[u] == 1 && visit[v] == 0) {
            visit[v] = 1;
            min_cost += min;

            nv++;
            printf("Edge %d - %d : (%d)\n", u, v, min);
        }
    }

    printf("Cost : %d\n", min_cost); // Prints minimum cost
}

int main() {
    int adj[10][10], N, i, j;
    printf("Enter number of nodes in the graph: ");
    scanf("%d", &N);
```

```
    printf("Enter the adjacency matrix\n");
    printf("Enter 0 for no connection and weights for connection\n");

    for (i = 0; i < N; i++) {
        for (j = 0; j < N; j++) {
            scanf("%d", &adj[i][j]);
            if (adj[i][j] == 0) {
                adj[i][j] = 999;
            }
        }
    }

    my_prim(adj, N);

    return 0;
}
```

Output:

```
Enter number of nodes in the graph: 4
Enter the adjacency matrix
Enter 0 for no connection and weights for connection
0
1 5 2 1 0 2 3 5 6 0 3 2 0 3 0
Edge 0 - 1 : (1)
Edge 1 - 2 : (2)
Edge 1 - 3 : (3)
Cost : 6
```

```
Enter number of nodes in the graph: 3
Enter the adjacency matrix
Enter 0 for no connection and weights for connection
0 10 1 10 0 6 1 6 0
Edge 0 - 2 : (1)
Edge 2 - 1 : (6)
Cost : 7
```

5. Design and implement a C/C++ Program to find Minimum Cost Spanning Tree of a given connected undirected graph using Kruskal's algorithm.

```
#include<stdio.h>
int ne=1,min_cost=0;
void main()
{
int n,i,j,min,a,u,b,v,cost[20][20],parent[20];
printf("Enter the no. of vertices:");
 scanf("%d",&n);
printf("\nEnter the cost matrix:\n");
for(i=1;i<=n;i++)
for(j=1;j<=n;j++)
scanf("%d",&cost[i][j]);
for(i=1;i<=n;i++)
parent[i]=0;
printf("\nThe edges of spanning tree are\n");
 while(ne<n)
{
min=999;
 for(i=1;i<=n;i++)
{
for(j=1;j<=n;j++)
{
if(cost[i][j]<min)
{
min=cost[i][j];
a=u=i;
b=v=j;
}
}
}
while(parent[u])
u=parent[u];
while(parent[v])
v=parent[v];
if(u!=v)
{
```

```
printf("Edge %d\t(%d->%d)=%d\n",ne++,a,b,min); min_cost=min_cost+min;
parent[v]=u;
}
cost[a][b]=cost[a][b]=999;
}
printf("\nMinimum cost=%d\n",min_cost);
}
```

//USING UNION AND FIND

```
#include <stdio.h>

int find(int x, int parent[]) {
    while (parent[x] >= 0){
        x = parent[x];
        printf("The value of %d", x);
    }
    return x;
}

void setUnion(int x, int y, int parent[]) {
    if (parent[x] < parent[y]) {
        parent[x] += parent[y];
        parent[y] = x;
    } else {
        parent[y] += parent[x];
        parent[x] = y;
    }
}

void kruskalAlgo(int edge[][3], int n) {
    int i, x, y, cost = 0, ecount = 0;
    int parent[n];
    for (i = 0; i < n; i++)
        parent[i] = -1;
    i = 0;
    int mst[n - 1][2];
    while (i < (n * n) && ecount < n - 1) {
        if (edge[i][2] == 999)
            break;
        x = find(edge[i][0], parent);
```

```
        y = find(edge[i][1], parent);
        //printf("\nThe x and y values are %d, %d",x,y);
        if (x != y) {
            cost += edge[i][2];
            mst[ecount][0] = edge[i][0];
            mst[ecount++][1] = edge[i][1];
            setUnion(x, y, parent);
        }
        i++;
    }
    if (ecount < n - 1)
        printf("The minimal spanning tree could not be found\n");
    else {
        printf("The minimal spanning tree cost: %d\n", cost);
        printf("The minimal spanning tree is:\n");
        for (i = 0; i < n - 1; i++)
            printf("%d - %d\n", mst[i][0], mst[i][1]);
    }
}

int main() {
    int n, i, j, k = 0;
    printf("Enter the number of vertices: ");
    scanf("%d", &n);
    int a[n][n];
    int edge[n * n][3];
    printf("Enter the adjacency matrix (999 - no edge):\n");
    for (i = 0; i < n; i++){
        for (j = 0; j < n; j++) {
            scanf("%d", &a[i][j]);
            edge[k][0] = i;
            edge[k][1] = j;
            edge[k++][2] = a[i][j];
        }
    }
```

```
    // Sorting the edges based on their weights
    for (i = 0; i < n * n; i++)
        for (j = 0; j < n * n - i - 1; j++)
            if (edge[j][2] > edge[j + 1][2]) {
                int temp[3];
                for (k = 0; k < 3; k++)
                    temp[k] = edge[j][k];
                for (k = 0; k < 3; k++)
                    edge[j][k] = edge[j + 1][k];
                for (k = 0; k < 3; k++)
                    edge[j + 1][k] = temp[k];
            }
    kruskalAlgo(edge, n);
    return 0;
}
```

```
Enter the number of vertices: 5
Enter the adjacency matrix (999 - no edge):
999 3 999 7 999 3 999 4 2 999 999 4 999 5 5 7 2 5 4 999 999 999 5 4 999
The minimal spanning tree cost: 13
The minimal spanning tree is:
1 - 3
0 - 1
1 - 2
4 - 3
```

Program 4: Design and implement C/C++ Program to find shortest paths from a given vertex in a weighted connected graph to other vertices using Dijkstra's algorithm.

```
#include<stdio.h>
#include<conio.h>
#define infinity 999
void dij(int n,int v,int cost[10][10],int dist[100])
{
int i,u,count,w,flag[10],min;
        for(i=1;i<=n;i++){
                flag[i]=0,dist[i]=cost[v][i];
        }
count=2;
while(count<=n)
```

```
{
        min=99;
           for(w=1;w<=n;w++){
                    if(dist[w]<min && !flag[w]){
                       min=dist[w],u=w;
                    }
           }
        flag[u]=1;
        count++;
                    for(w=1;w<=n;w++){
                             if((dist[u]+cost[u][w]<dist[w]) && !flag[w]){
                             dist[w]=dist[u]+cost[u][w];
                    }
   }
}
}
void main()
{
int n,v,i,j,cost[10][10],dist[10];

printf("\n Enter the number of nodes:");
scanf("%d",&n);
printf("\n Enter the cost matrix:\n");
        for(i=1;i<=n;i++)
        for(j=1;j<=n;j++)
        {
        scanf("%d",&cost[i][j]);
        if(cost[i][j]==0)
        cost[i][j]=infinity;
        }
printf("\n Enter the source matrix:");
scanf("%d",&v);
dij(n,v,cost,dist);
printf("\n Shortest path:\n");
        for(i=1;i<=n;i++)
        if(i!=v)
        printf("%d->%d,cost=%d\n",v,i,dist[i]);

}
```

```
 Enter the number of nodes:5

 Enter the cost matrix:
0 3 0 7 0 3 0 4 2 0 0 4 0 5 5 7 2 5 0 4 0 0 5 4 0

 Enter the source matrix:1

 Shortest path:
1->2,cost=3
1->3,cost=7
1->4,cost=5
1->5,cost=9
```

```
 Enter the number of nodes:5

 Enter the cost matrix:
0 3 0 7 0 3 0 4 2 0 0 4 0 5 5 7 2 5 0 4 0 0 5 4 0

 Enter the source matrix:4

 Shortest path:
4->1,cost=5
4->2,cost=2
4->3,cost=5
4->5,cost=4
```

3. b. Design and implement C/C++ Program to find the transitive closure using Warshal's algorithm.

```
#include<stdio.h>
void warshall(int[10][10],int);
void main()
{
int a[10][10],i,j,n;

printf("Enter the number of nodes:");
scanf("%d",&n);
printf("\nEnter the adjacency matrix:\n");
for(i=1;i<=n;i++)
for(j=1;j<=n;j++)
scanf("%d",&a[i][j]);
printf("The adjacency matirx is:\n");
for(i=1;i<=n;i++)
{
for(j=1;j<=n;j++)
{
printf("%d\t",a[i][j]);
}
printf("\n");
}
warshall(a,n);

}
void warshall(int p[10][10],int n)
{
int i,j,k;
for(k=1;k<=n;k++)
```

```
{
for(j=1;j<=n;j++)
{
for(i=1;i<=n;i++)
{
if((p[i][j]==0) && (p[i][k]==1) && (p[k][j]==1))
{
p[i][j]=1;
}
}
}
}
printf("\nThe path matrix is:\n");
for(i=1;i<=n;i++)
{
for(j=1;j<=n;j++)
{
printf("%d\t",p[i][j]);
}
printf("\n");
}
}
```

OUTPUT

Enter the number of nodes:4

Enter the adjacency matrix:
0 1 0 0
0 0 0 1
0 0 0 0

1 0 1 0

The adjacency matirx is:

0	**1**	**0**	**0**
0	**0**	**0**	**1**
0	**0**	**0**	**0**
1	**0**	**1**	**0**

The path matrix is:

1	**1**	**1**	**1**
1	**1**	**1**	**1**
0	**0**	**0**	**0**
1	**1**	**1**	**1**

a. Design and implement C/C++ Program to solve All-Pairs Shortest Paths problem using Floyd's algorithm.

```
#include<stdio.h>


int min(int a,int b)
{
   return(a<b)?a:b;
}
void floyd(int p[10][10],int n)
{
   int i,j,k;
      for(k=1;k<=n;k++)
         for(i =1;i<=n;i++)
            for(j=1;j<=n;j++)
               p[i][j]=min(p[i][j],p[i][k]+p[k][j]);
}
void main()
{
   int a[10][10],n,i,j;
```

```
    printf("\nEnter the n value:");
    scanf("%d",&n);
    printf(" \nEnter the graph data:\n");
    for(i=1;i<=n;i++)
        for(j=1;j<=n;j++)
            scanf("%d",&a[i][j]);
    floyd(a,n);
    printf(" \nShortest path matrix \n");
    for(i=1;i<=n;i++)
    {
        for(j=1;j<=n;j++)
            printf("%d ",a[i][j]);
            printf("\n");
    }

}
```

5 Design and implement C/C++ Program to obtain the Topological ordering of vertices in a given digraph.

```
#include<stdio.h>

int a[10][10],n,indegre[10];
void find_indegre(){
    int j,i,sum;
    for(j=0;j<n;j++)
        {
        sum=0;
        for(i=0;i<n;i++)
        sum+=a[i][j];
        indegre[j]=sum;
        }
}
void topology()
{
```

```
    int i,u,v,t[10],s[10],top=-1,k=0;
    find_indegre();
    for(i=0;i<n;i++)
    {
    if(indegre[i]==0) s[++top]=i;
    }
    while(top!=-1)
    {
    u=s[top--];
    t[k++]=u;
        for(v=0;v<n;v++)
        {
            if(a[u][v]==1)
            {
            indegre[v]--;
            if(indegre[v]==0) s[++top]=v;
            }
        }
    }
printf("The topological Sequence is:\n");
    for(i=0;i<n;i++)
    printf("%d ",t[i]);
}
void main()
{
    int i,j;

    printf("Enter number of nodes:");
    scanf("%d",&n);
    printf("\nEnter the adjacency matrix:\n");
        for(i=0;i<n;i++)
        {
        for(j=0;j<n;j++)
        scanf("%d",&a[i][j]);
        }
    topology();

}
```

```
Enter number of nodes:5

Enter the adjacency matrix:
0 0 1 0 0
0 0 1 0 0
0 0 0 1 1
0 0 0 0 1
0 0 0 0 0
The topological Sequence is:
1 0 2 3 4
```

8 Design and implement C/C++ Program to find a subset of a given set S = {sl , s2,.....,sn} of n positive integers whose sum is equal to a given positive integer d.

```
#include<stdio.h>

int s[10] , x[10],d ;
void sumofsub ( int , int , int ) ;
void main ()
{
int n , sum = 0 ;
int i ;

printf ( " \n Enter the size of the set : " ) ;
scanf ( "%d" , &n ) ;
printf ( " \n Enter the set in increasing order:\n" ) ;
for ( i = 1 ; i <= n ; i++ )
scanf ("%d", &s[i] ) ;
printf ( " \n Enter the value of d : \n " ) ;
scanf ( "%d" , &d ) ;
for ( i = 1 ; i <= n ; i++ )
sum = sum + s[i] ;
if ( sum < d || s[1] > d )
printf ( " \n No subset possible : " ) ;
```

```
else
sumofsub ( 0 , 1 , sum ) ;
}
void sumofsub ( int m , int k , int r )
{
int i=1 ;
x[k] = 1 ;
if ( ( m + s[k] ) == d )
{
printf("Subset:");
for ( i = 1 ; i <= k ; i++ )
if ( x[i] == 1 )
printf ( "\t%d" , s[i] ) ;
printf ( "\n" ) ;
}
else
if ( m + s[k] + s[k+1] <= d )
sumofsub ( m + s[k] , k + 1 , r - s[k] ) ;
if ( ( m + r - s[k] >= d ) && ( m + s[k+1] <=d ) )
{
x[k] = 0;
sumofsub ( m , k + 1 , r - s[k] ) ;
}
}
```

```
Enter the size of the set : 4

Enter the set in increasing order:
3 5 6 7

Enter the value of d :
15
Subset: 3       5       7
```

6 Design and implement C/C++ Program to solve 0/1 Knapsack problem using Dynamic Programming method.

```
#include<stdio.h>

int w[10],p[10],v[10][10],n,i,j,cap,x[10]={0};
int max(int i,int j)
   {
   return ((i>j)?i:j);
   }
int knap(int i,int j)
{
   int value;
      if(v[i][j]<0)
      {
         if(j<w[i])
            value=knap(i-1,j);
         else
            value=max(knap(i-1,j),p[i]+knap(i-1,j-w[i]));
```

```
            v[i][j]=value;
        }
    return(v[i][j]);
}
void main()
{
int profit,count=0;

printf("\nEnter the number of elements\n");
scanf("%d",&n);
printf("Enter the profit and weights of the elements\n");
    for(i=1;i<=n;i++)
    {
    printf("For item no %d\n",i);
    scanf("%d%d",&p[i],&w[i]);
    }
    printf("\nEnter the capacity \n");
    scanf("%d",&cap);
        for(i=0;i<=n;i++)
        for(j=0;j<=cap;j++)
            if((i==0)||(j==0))
                v[i][j]=0;
            else
                v[i][j]=-1;
    profit=knap(n,cap);
    i=n;
    j=cap;
    while(j!=0&&i!=0)
    {
    if(v[i][j]!=v[i-1][j])
        {
        x[i]=1;
        j=j-w[i];
        i--;
        }
    else
    i--;
    }
    printf("Items included are\n");
    printf("Sl.no\tweight\tprofit\n");
```

```
    for(i=1;i<=n;i++)
        if(x[i])
            printf("%d\t%d\t%d\n",++count,w[i],p[i]);
  printf("Total profit = %d\n",profit);

}
```

```
Enter the number of elements
4
Enter the profit and weights of the elements
For item no 1
12
2
For item no 2
10
1
For item no 3
20
3
For item no 4
15
2

Enter the capacity
5
Items included are
Sl.no   weight  profit
1       2       12
2       1       10
3       2       15
Total profit = 37
```

Design and implement C/C++ Program to solve discrete Knapsack and continuous Knapsack problems using greedy approximation method.

```
#include<stdio.h>
void knapsack(int n, float weight[], float profit[], float capacity)
{
  float x[20], tp = 0;
  int i, j, u;
  u = capacity;

  for (i = 0; i < n; i++)
    x[i] = 0.0;

  for (i = 0; i < n; i++) {
    if (weight[i] > u)
      break;
    -else {
      x[i] = 1.0;
      tp = tp + profit[i];
      u = u - weight[i];
    }
  }

  if (i < n)
    x[i] = u / weight[i];

  tp = tp + (x[i] * profit[i]);

  printf("\nThe result vector is:- ");
  for (i = 0; i < n; i++)
    printf("%f\t", x[i]);
```

```
  printf("\nMaximum profit is:- %f", tp);

}

void Dknapsack(int n, float weight[], float profit[], float capacity)
{
  float x[20], tp = 0;
  int i, j, u;
  u = capacity;

  for (i = 0; i < n; i++)
    x[i] = 0.0;

  for (i = 0; i < n; i++) {
    if (weighft[i] > u)
      continue;
    else {
      x[i] = 1.0;
      tp = tp + profit[i];
      u = u - weight[i];
    }
  }


  printf("\nThe result vector is:- ");
  for (i = 0; i < n; i++)
    printf("%f\t", x[i]);

  printf("\nMaximum profit is:- %f", tp);

}

int main() {
  float weight[20], profit[20], capacity;
  int num, i, j;
  float ratio[20], temp;

  printf("\nEnter the no. of objects:- ");
  scanf("%d", &num);
```

```
  printf("\nEnter the wts of each object:- ");
  for (i = 0; i < num; i++) {
    scanf("%f", &weight[i]);
  }
   printf("\nEnter the profits of each object:- ");
  for (i = 0; i < num; i++) {
    scanf("%f", &profit[i]);
  }

  printf("\nEnter the capacity of knapsack:- ");
  scanf("%f", &capacity);

  for (i = 0; i < num; i++) {
    ratio[i] = profit[i] / weight[i];
  }

  for (i = 0; i < num; i++) {
    for (j = i + 1; j < num; j++) {
      if (ratio[i] < ratio[j]) {
        temp = ratio[j];
        ratio[j] = ratio[i];
        ratio[i] = temp;

        temp = weight[j];
        weight[j] = weight[i];
        weight[i] = temp;

        temp = profit[j];
        profit[j] = profit[i];
        profit[i] = temp;
      }
    }
  }

  knapsack(num, weight, profit, capacity);
  Dknapsack(num, weight, profit, capacity);
  return(0);
}
```

NQueens Program:

```
#include<stdio.h>

#include<stdlib.h>
#define MAX 50
int can_place(int c[],int r)
{
int i;
for(i=0;i<r;i++)
if(c[i]==c[r] || (abs(c[i]-c[r])==abs(i-r)))
return 0;
return 1;
}
void display(int c[],int n)
{
int i,j;
char cb[10][10];
for(i=0;i<n;i++)
for(j=0;j<n;j++)
cb[i][j]='-';
for(i=0;i<n;i++)
cb[i][c[i]]='Q';
```

```
for(i=0;i<n;i++)
{
for(j=0;j<n;j++)
printf("%c",cb[i][j]);
printf("\n");

}
}
void n_queens(int n)
{
int r;
int c[MAX];
c[0]=-1;
r=0;
while(r>=0)
{ c[r]++;
while(c[r]<n && !can_place(c,r))
c[r]++;
if(c[r]<n)
{ if(r==n-1)
{ display(c,n);
printf("\n\n");
}
else
{ r++;
c[r]=-1;
}
}
else
r--;
}
}
void main()
{
int n;

printf("\nEnter the no. of queens:");
scanf("%d",&n);
n_queens(n);
```

}